Hybrid Neighborhood Evaluation in Tabu Search and Simulated Annealing for the Permutation Flow Shop Problem

Remigiusz Wojewódzki[1] and Wojciech Bożejko[2]

[1] Wrocław University of Science and Technology, Poland
remigiusz.wojewodzki@gmail.com
[2] Wrocław University of Science and Technology, Poland
wojciech.bozejko@pwr.edu.pl

Abstract. We investigate the impact of three neighborhood definitions on the performance of two classical metaheuristics (Tabu Search and Simulated Annealing) for the permutation Flow Shop Scheduling Problem (FSP) under the makespan minimization objective. Besides the standard adjacent swap neighborhood, we study (i) a composite disjoint-adjacent swap construction obtained via a dynamic programming (DP) selection of a set of non-overlapping swaps minimizing the cumulative makespan delta, and (ii) a recursive dynasearch-inspired neighborhood enabling deeper local restructuring. We additionally introduce iteration-wise logging and optional temperature reheating (time-based stagnation triggers) to analyze convergence dynamics. Experiments on Taillard benchmark instances show that the composite ('Fibonacci-inspired') neighborhood accelerates early descent but tends to plateau once all single adjacent improvements vanish, while dynasearch continues to find greater improvements at a higher computational cost per iteration. We detail the trade-offs among exploration breadth, per-iteration computational overhead, and convergence stability.

Keywords: Flow shop scheduling · Metaheuristics · Tabu search · Simulated annealing · Composite neighborhood · Dynasearch

1 Introduction

The permutation Flow Shop Scheduling Problem (FSP) seeks an ordering (permutation) of n jobs processed on m machines in identical order, minimizing the makespan $C_{\max}$. The problem is NP-hard for $m \geq 3$ and has motivated extensive research into neighborhood-based heuristic and metaheuristic methods [2,9,15]. Tabu Search, originally proposed by Glover [7,8], and Simulated Annealing, introduced by Kirkpatrick et al. [10], remain among the most widely used metaheuristic frameworks for combinatorial optimization.

This paper revisits two standard metaheuristics—Tabu Search (TS) and Simulated Annealing (SA)—through the lens of *hybrid neighborhood design*. Beyond the ubiquitous adjacent swap neighborhood, we implement and compare:

– A composite disjoint-adjacent swap neighborhood (internally nicknamed *Fibonacci*) built via dynamic programming to select a set of non-overlapping improving (or minimally degrading) adjacent swaps.
– A dynasearch-inspired recursive neighborhood allowing deeper structured permutations.

We empirically examine how these neighborhoods alter convergence speed, stagnation behavior, and final solution quality under identical time budgets. We deliberately employ these two classical frameworks as standardized testbeds, so that observed performance differences can be attributed to the neighborhood design rather than to algorithmic sophistication.

Contributions.

1. A unified experimental framework (with per-iteration logging) comparing three neighborhoods inside TS and SA under common timing constraints.
2. A DP-based composite adjacent neighborhood balancing local intensification and mild diversification.
3. A time-based reheating mechanism for SA (temperature floor plus stagnation-triggered multiplicative reheats).
4. An empirical analysis on Taillard instances highlighting (i) rapid early exploitation by the composite neighborhood, (ii) its susceptibility to plateau once all single-swap improvements disappear, and (iii) the complementary depth of the dynasearch neighborhood.

The ultimate objective of this research is to formulate a comprehensive conclusion on the relative efficiency and robustness of the analyzed neighborhood structures with respect to the imposed computational time limits and the scale of problem instances, thereby providing deeper insights into their algorithmic behavior and practical applicability.

2 Problem Definition

Given processing time matrix $P = [p_{k,j}]_{m\times n}$ for machine $k = 1, \ldots, m$ and job $j = 1, \ldots, n$, a permutation π defines the job order. Permutations and their interactions can also be analyzed using positive definite functions on permutation groups, providing a theoretical foundation for studying correlations between job sequences [3]. Let $C_{k,j}$ be the completion time of job j (in π-order) on machine k fulfilling equalities:

$$C_{1,j} = C_{1,j-1} + p_{1,\pi(j)}, \quad C_{k,1} = C_{k-1,1} + p_{k,\pi(1)},$$

$$C_{k,j} = \max\{C_{k,j-1},\, C_{k-1,j}\} + p_{k,\pi(j)}, \quad \text{for } k = 1, \ldots, m,\ j = 1, \ldots, n. \tag{1}$$

Makespan $C_{\max} = C_{m,n}$ should be minimized. The objective of the permutation Flow Shop Scheduling Problem is to find a permutation π that *minimizes the makespan* $C_{\max}$, i.e., the completion time of the last job on the last machine when the jobs are processed in the order given by π.

3 Neighborhood Structures

Following [15], a *neighborhood* of a solution x is defined as the set of all solutions obtainable by applying a specific, allowed modification to x. It determines how the solution space is explored and exploited during the search process.

Building on the work of Ahuja et al.[1,8], general methods for constructing Very Large-Scale Neighborhoods (VLSN) can be identified. These techniques include generating new solutions by combining elements from multiple reference solutions, which enables exploration of larger areas of the solution space. Another approach employs dynamic programming to accelerate computations and efficiently generate neighborhoods for large datasets. A key component is the creation of improvement graphs, which allow rapid assessment of promising moves. Finally, VLSN methods are often integrated with other metaheuristic techniques, such as Tabu Search or Simulated Annealing, resulting in iterative algorithms capable of effectively searching for high-quality or optimal solutions.

Adjacent Swap (Baseline) Each neighbor swaps positions $(i, i{+}1)$ for $i = 1, \ldots, n-1$. Its per-iteration evaluation cost is low, enabling many iterations but shallow local moves [11,15].

Composite Disjoint-Adjacent (in the rest of the paper, we will refer to this neighborhood as the Fibonacci*) neighborhood due to the size of its structure.* Starting from π, we enumerate all single adjacent swaps, computing $\Delta_i = C_{\max}(\pi^{(i)}) - C_{\max}(\pi)$. A dynamic program selects a subset of non-overlapping indices minimizing the total $\sum \Delta_i$. If no negative Δ_i exists, the current implementation either (i) chooses the least positive swap or (ii) halts (variant under discussion). The resulting composite move may apply multiple disjoint swaps simultaneously, approximating a bounded multi-step local descent in one iteration. The theoretical background and properties of such swap-based multimove strategies were discussed by Bożejko and Wodecki [4].

Complexity. Enumerating $(n{-}1)$ swaps requires $(n{-}1)$ recomputations of $C_{\max}$ (each $O(mn)$), plus $O(n)$ DP. Overall, per iteration $O(mn^2)$ versus $O(mn)$ for a single adjacent swap evaluation.

Dynasearch-Inspired Recursive Neighborhood Dynasearch explores structured sequences of non-conflicting moves [5] (e.g., path decompositions) at increased computational effort, often reaching permutations unreachable by a limited number of adjacent steps. The method considers all segments $[i, j]$ of the permutation and swaps their endpoints $\pi(i) \leftrightarrow \pi(j)$, leaving the interior unchanged. A dynamic programming procedure selects a set of non-overlapping improving segments, minimizing the total makespan delta. The number of candidate segments is $O(n^2)$, yielding per-iteration complexity $O(mn^3)$.

Six-Operator Neighborhood Deroussi et al. [6] define the neighborhood of a solution as the set of solutions obtained by applying six operators that transform the current solution. These operators were designed specifically for the permutation Flow Shop Problem within a Tabu Search framework. They include, among others, single-job moves, block moves, multi-element shifts, and combinations of these operations. Compared to classical approaches (e.g., adjacent job swaps), this strategy enables faster and more comprehensive exploration of the solution space, increasing the likelihood of finding improved permutations.

Global-Local Neighborhood Serna and Gambardella [12] describe a global-local neighborhood for the permutation Flow Shop Problem. This neighborhood is composed of two types of moves. The first type, Nopt1, involves swapping two jobs or relocating a single job within the permutation. According to Mastrolilli & Gambardella, for a given job j, Nopt1 generates neighboring permutations by moving j to each possible position in the sequence, enabling quick local improvements. The second type consists of larger, structured perturbations that move entire blocks of jobs, resulting in more significant, global changes in the permutation. Combining these two types allows the search to escape shallow local optima: small Nopt1 moves provide fast local refinement, while larger perturbations enable the exploration of regions in the solution space that are inaccessible through minor changes.

4 Metaheuristics

Based on Glover and Laguna [8] and Blum and Roli [2], metaheuristics are high-level algorithmic frameworks guiding the search for an optimal solution. They are not defined for a specific problem but rather provide a general description and a set of strategies for exploring and exploiting the solution space effectively.

Tabu Search (TS). First formalized by Glover [7] maintains a tabu list, following the guidelines presented by Smutnicki [13] for the flow shop problem, keyed either by primitive moves (adjacent swap indices) or composite signatures (for *Fibonacci* / Dynasearch). A tenure T_{Tabu} forbids revisits unless an aspiration criterion ($C_{\max}$ improvement) holds. At each iteration, the best admissible neighbor is selected.

Simulated Annealing (SA) Following the framework of Kirkpatrick et al. [10], We follow Smutnicki's [13] recommendations for iterative improvement strategies while using multiplicative cooling $T \leftarrow \alpha T$ with floor $T_{\text{floor}} = f_{\text{floor}} \cdot T_{\text{final}}$ and optional time-based reheating: if no improvement for τ_{stagn} milliseconds and $T < T_{\text{init}}$, then $T \leftarrow \min(T_{\text{init}}, r \cdot T)$. Acceptance probability standard:

$$\Pr(\mathit{accept}) = \begin{cases} 1 & \text{for } \Delta < 0, \\ \exp(-\Delta/T) & \text{otherwise.} \end{cases} \tag{2}$$

5 Experimental Setup

All experiments were conducted on a MacBook Pro with an M4 Pro processor, 24 GB of RAM, running macOS 26.0.1 (25A362) and Python 3.11.

Benchmarks Taillard instances [14] of varying (n, m) (e.g., $n \in \{50, 100, 200, 500\}$ and $m \in \{10, 20\}$) were used. Each run was constrained by a wall-clock time budget (e.g., 100 s). Seeds are fixed for reproducibility.

Parameters The algorithms were configured with the following parameters: tabu tenure equal to 10; simulated annealing with $T_{\text{init}} = 1000$, $T_{\text{final}} = 1$, cooling rate $\alpha = 0.95$, reheating factor $r = 1.5$, and a stagnation threshold of 500 ms.

Metrics The evaluation considered the best $C_{\max}$ obtained over time, convergence curves for different neighborhood strategies, the number of effective moves per second (normalized), and the plateau detection latency. The choice of experimental parameters and metrics was primarily aimed at assessing the impact of different neighborhood structures on the final solution quality rather than evaluating the absolute performance of the algorithms.

6 Large-Scale Experimentation

For each tested instance, we observe that under the assumed time limits, the Dynasearch neighborhood is often unable to complete even a single iteration. Therefore, its results should be excluded from the final comparative conclusions. For small-scale datasets, however, Dynasearch is capable of finding the best solution within the upper time limits, even when the other two neighborhoods become trapped in local optima. Nevertheless, for larger instances, Dynasearch frequently fails to execute a single iteration within the predefined time budget, which significantly limits its practical applicability in short-run scenarios.

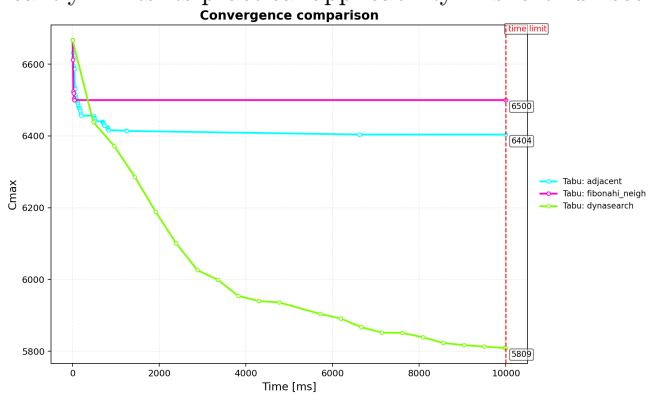

Fig. 1: Small dataset, extended time limit.

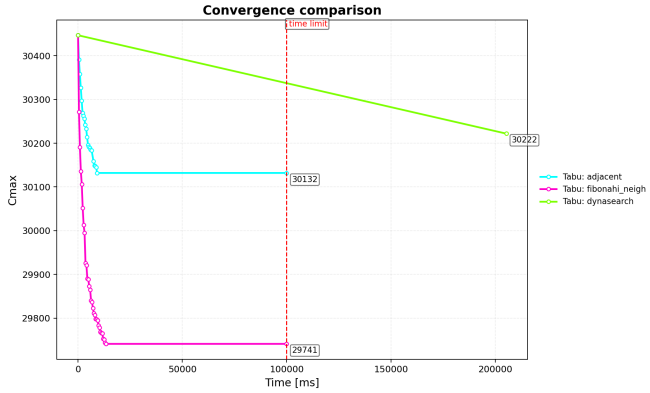

Fig. 2: Large dataset, extended time limit.

7 Results and Discussion

The comparison between the *Fibonacci*, Adjacent, and Dynasearch approaches reveals distinct behavioral and performance characteristics. The composite neighborhood outperforms the adjacent neighborhood in the early search phase (showing a steeper initial drop in $C_{\max}$) due to its ability to combine non-overlapping improvements. However, once all single adjacent deltas become non-negative, the composite move degenerates to size 1 and may stagnate in local minima.

For larger datasets (above 200×10), the *Fibonacci* method finds better solutions within the same time limits compared to Adjacent. In contrast, Dynasearch achieves further and often the best improvements, but at a much higher computational cost. Depending on the dataset size, a single iteration of Dynasearch may require up to 200 times more time than the predefined limit. This indicates that Dynasearch can deliver the best solutions only when provided with sufficiently large time budgets, which grow rapidly with input size.

During such extended computations, the remaining two approaches (*Fibonacci* and Adjacent) tend to get trapped in local optima, losing their improvement potential. In these cases, time-triggered reheating in Simulated Annealing (SA) occasionally enables the algorithm to escape from shallow plateaus, restoring diversification and allowing further exploration of the search space.

The average results presented in Tables 1–6 include the relative percentage deviation (PRD) of the makespan, defined as:

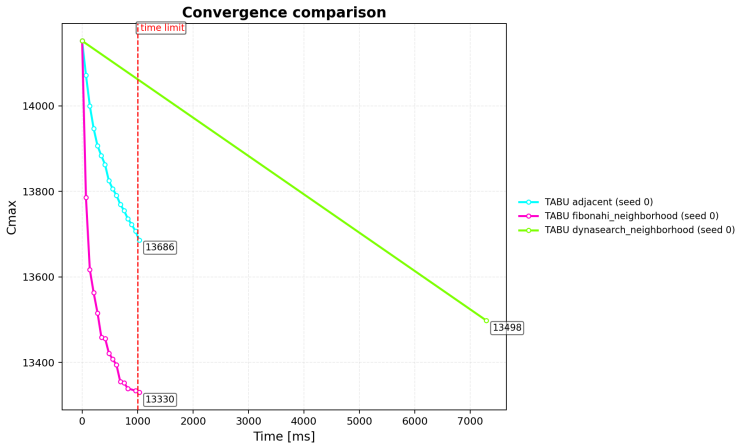

Fig. 3: 200 jobs, 20 machines, 1000 ms limit.

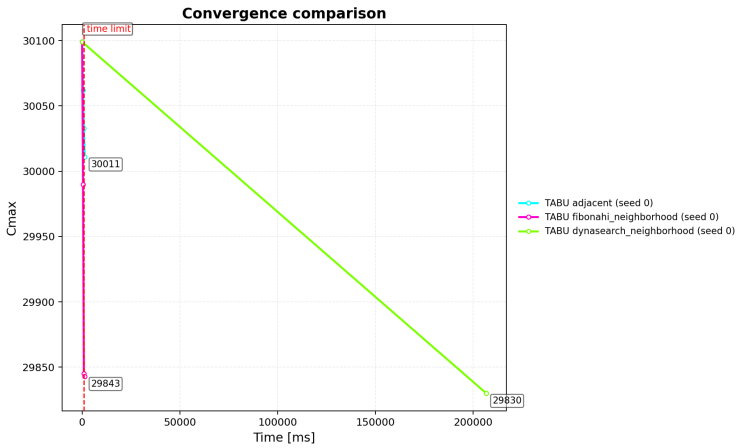

Fig. 4: 500 jobs, 20 machines, 1000 ms limit.

$$\mathrm{PRD} = \frac{C_{\max}^{\text{obtained}} - C_{\max}^{\text{optimal}}}{C_{\max}^{\text{optimal}}} \cdot 100\%, \tag{3}$$

where $C_{\max}^{\text{obtained}}$ is the makespan obtained by the heuristic method and $C_{\max}^{\text{optimal}}$ is the best known or optimal makespan. These PRD values summarize the average performance of each method over all instances for a given time limit, highlighting which approaches are closer to the optimal solution. Note that Dynasearch has been omitted from the tables due to its tendency to exceed the time limit in a single iteration, especially for larger instances, making its inclusion impractical for the standardized time-limited comparison.

8 Conclusions and Future Work

Hybrid composite neighborhoods accelerate early exploitation in Tabu Search (TS) and Simulated Annealing (SA) but risk stagnation without diversification. Dynasearch remains effective but is significantly more computationally expensive. Future work should focus on incremental evaluation, adaptive switching between neighborhood structures, probabilistic variant sampling, and exploring alternative neighborhood definitions with comparative analysis of their performance and computational cost. Future work will also integrate the proposed neighborhoods into more advanced solvers (e.g., iterated local search, variable neighborhood descent) and include comparisons with state-of-the-art methods.

Time limit: $tl_{\text{ms}} = 100$

file	n	m	tl_{ms}	fib_{TABU}	fib_{SA}	dyn_{TABU}	dyn_{SA}	adj_{TABU}	adj_{SA}
tai20_5	20	5	100	17.39	17.39	6.18	6.18	**7.18**	7.98
tai20_10	20	10	100	29.73	29.73	14.68	14.68	**19.11**	20.72
tai20_20	20	20	100	36.37	36.37	24.83	24.83	31.85	**30.28**
tai50_5	50	5	100	12.13	12.13	5.69	5.89	10.71	**7.93**
tai50_10	50	10	100	20.23	20.23	16.84	16.97	**19.65**	22.10
tai50_20	50	20	100	32.20	**32.19**	31.14	31.14	32.54	34.23
tai100_5	100	5	100	**11.15**	11.15	11.01	11.01	11.80	11.97
tai100_10	100	10	100	**17.34**	17.34	17.33	17.33	18.41	19.62
tai100_20	100	20	100	**24.50**	24.50	25.50	25.50	27.55	27.94
tai200_10	200	10	100	**14.01**	14.01	13.20	13.20	15.40	15.28
tai200_20	200	20	100	**21.62**	21.62	20.71	20.71	23.26	23.33
tai500_20	500	20	100	**15.57**	15.57	14.51	14.51	16.25	16.03
Average			100	21.02	21.02	16.80	16.83	**19.48**	19.78

Table 1: Average results for $T_{\text{limit}} = 100$ ms.

Time limit: $tl_{\text{ms}} = 500$

file	n	m	tl_{ms}	fib_{TABU}	fib_{SA}	dyn_{TABU}	dyn_{SA}	adj_{TABU}	adj_{SA}
tai20_5	20	5	500	17.39	17.39	6.18	6.18	**5.23**	6.52
tai20_10	20	10	500	29.73	29.73	14.68	14.68	**14.65**	20.33
tai20_20	20	20	500	36.37	36.37	24.83	24.83	**27.14**	30.22
tai50_5	50	5	500	12.13	12.13	2.95	2.95	10.10	**3.54**
tai50_10	50	10	500	20.23	20.23	9.94	9.94	**17.97**	12.69
tai50_20	50	20	500	32.19	32.19	23.31	23.31	30.55	**26.58**
tai100_5	100	5	500	11.15	11.15	8.73	8.73	10.89	**6.68**
tai100_10	100	10	500	17.19	17.19	15.85	15.98	**16.91**	17.16
tai100_20	100	20	500	**23.02**	**23.02**	25.50	25.50	24.94	26.33
tai200_10	200	10	500	**12.73**	**12.73**	13.20	13.20	14.10	14.72
tai200_20	200	20	500	**19.74**	19.79	20.71	20.71	22.14	22.80
tai500_20	500	20	500	15.13	15.13	14.51	14.51	**16.09**	16.00
Average			500	20.58	20.59	15.03	15.04	**17.56**	16.96

Table 2: Average results for $T_{\text{limit}} = 500$ ms.

Time limit: $tl_{\text{ms}} = 1000$

file	n	m	tl_{ms}	fib_{TABU}	fib_{SA}	dyn_{TABU}	dyn_{SA}	adj_{TABU}	adj_{SA}
tai20_5	20	5	1000	17.39	17.39	6.18	6.18	**4.76**	6.31
tai20_10	20	10	1000	29.73	29.73	14.68	14.68	**13.34**	20.25
tai20_20	20	20	1000	36.37	36.37	24.83	24.83	**25.56**	30.22
tai50_5	50	5	1000	12.13	12.13	2.95	2.95	9.84	**2.44**
tai50_10	50	10	1000	20.23	20.23	7.46	7.46	**17.82**	11.81
tai50_20	50	20	1000	32.19	32.19	20.22	20.22	**29.27**	24.92
tai100_5	100	5	1000	11.15	11.15	7.04	7.04	10.83	**4.41**
tai100_10	100	10	1000	17.19	17.19	14.03	14.22	**16.68**	12.11
tai100_20	100	20	1000	**23.02**	**23.02**	23.43	23.43	24.06	25.41
tai200_10	200	10	1000	**12.71**	**12.71**	13.20	13.20	13.44	14.32
tai200_20	200	20	1000	**19.13**	19.11	20.71	20.71	21.46	22.49
tai500_20	500	20	1000	14.94	14.94	14.51	14.51	**15.96**	15.93
Average			1000	20.52	20.51	14.10	14.12	**16.92**	15.89

Table 3: Average results for $T_{\text{limit}} = 1000$ ms.

Time limit: $tl_{\text{ms}} = 2000$

file	n	m	tl_{ms}	fib_{TABU}	fib_{SA}	dyn_{TABU}	dyn_{SA}	adj_{TABU}	adj_{SA}
tai20_5	20	5	2000	17.39	17.39	6.18	6.18	**4.40**	6.31
tai20_10	20	10	2000	29.73	29.73	14.68	14.68	**13.02**	19.80
tai20_20	20	20	2000	36.37	36.37	24.83	24.83	**24.73**	30.22
tai50_5	50	5	2000	12.13	12.13	2.95	2.95	9.80	**1.96**
tai50_10	50	10	2000	20.23	20.23	6.95	6.95	**17.33**	10.94
tai50_20	50	20	2000	32.19	32.19	16.55	16.52	**28.41**	24.61
tai100_5	100	5	2000	11.15	11.15	4.47	4.58	10.81	**3.32**
tai100_10	100	10	2000	17.19	17.19	11.25	10.98	**16.53**	9.62
tai100_20	100	20	2000	**23.02**	**23.02**	21.31	21.31	23.62	19.63
tai200_10	200	10	2000	**12.71**	**12.71**	13.20	13.20	13.27	13.77
tai200_20	200	20	2000	**18.65**	18.65	20.71	20.71	20.70	21.80
tai500_20	500	20	2000	14.48	14.48	14.51	14.51	**15.75**	15.91
Average			2000	20.44	20.44	13.13	13.12	**16.53**	14.82

Table 4: Average results for $T_{\text{limit}} = 2000$ ms.

Time limit: $tl_{\text{ms}} = 5000$

file	n	m	tl_{ms}	fib_{TABU}	fib_{SA}	dyn_{TABU}	dyn_{SA}	adj_{TABU}	adj_{SA}
tai20_5	20	5	5000	17.39	17.39	6.18	6.18	**4.17**	6.24
tai20_10	20	10	5000	29.73	29.73	14.68	14.68	**12.58**	19.58
tai20_20	20	20	5000	36.37	36.37	24.83	24.83	**22.59**	30.36
tai50_5	50	5	5000	12.13	12.13	2.95	2.95	9.38	**2.21**
tai50_10	50	10	5000	20.23	20.23	6.95	6.95	**17.10**	9.72
tai50_20	50	20	5000	32.19	32.19	14.80	14.84	**27.56**	24.44
tai100_5	100	5	5000	11.15	11.15	2.34	2.34	10.72	**2.40**
tai100_10	100	10	5000	17.19	17.19	6.57	6.48	**16.52**	8.27
tai100_20	100	20	5000	**23.02**	**23.02**	17.79	17.70	23.20	17.52
tai200_10	200	10	5000	**12.71**	**12.71**	13.20	13.20	13.19	8.53
tai200_20	200	20	5000	**18.60**	18.60	20.71	20.71	19.83	20.20
tai500_20	500	20	5000	13.91	13.90	14.51	14.51	**15.34**	15.82
Average			5000	20.39	20.38	12.13	12.11	**16.02**	13.77

Table 5: Average results for $T_{\text{limit}} = 5000$ ms.

Time limit: $tl_{\text{ms}} = 10000$

file	n	m	tl_{ms}	fib_{TABU}	fib_{SA}	dyn_{TABU}	dyn_{SA}	adj_{TABU}	adj_{SA}
tai20_5	20	5	10 000	17.39	17.39	6.18	6.18	**4.04**	6.24
tai20_10	20	10	10 000	29.73	29.73	14.68	14.68	**12.21**	19.58
tai20_20	20	20	10 000	36.37	36.37	24.83	24.83	**22.47**	30.16
tai50_5	50	5	10 000	12.13	12.13	2.95	2.95	9.38	**1.93**
tai50_10	50	10	10 000	20.23	20.23	6.95	6.95	**17.10**	9.07
tai50_20	50	20	10 000	32.19	32.19	14.73	14.80	**26.58**	24.32
tai100_5	100	5	10 000	11.15	11.15	1.93	1.93	10.57	**2.37**
tai100_10	100	10	10 000	17.19	17.19	4.11	4.11	**16.50**	6.17
tai100_20	100	20	10 000	**23.02**	**23.02**	15.42	15.42	22.93	17.01
tai200_10	200	10	10 000	**12.71**	**12.71**	11.66	11.66	13.16	7.23
tai200_20	200	20	10 000	**18.60**	18.60	19.32	19.32	19.63	14.59
tai500_20	500	20	10 000	13.62	13.62	14.51	14.51	**14.97**	15.53
Average			10 000	20.36	20.36	11.44	11.45	**15.80**	12.85

Table 6: Average results for $T_{\text{limit}} = 10000$ ms.

Acknowledgements

The authors acknowledge the use of Large Language Models (OpenAI GPT-5) for auxiliary tasks, including linguistic polishing of the manuscript, preliminary literature retrieval, automation of selected experimental processes (e.g., data preprocessing and log analysis), discussion of research ideas, and support in generating experimental code. The models were not involved in the creation of core research concepts or the interpretation of the obtained results.

A Pseudocode Appendix: Metaheuristics and Neighborhoods for the Permutation Flow Shop

Let m be the number of machines, n the number of jobs, and $P = (p_{k,j})$ processing times. A permutation (sequence) is π. The objective is to minimize the makespan $C_{\max}(\pi)$. A function $\textsc{Makespan}(\pi)$ returns $C_{\max}(\pi)$. Time budget in milliseconds: T_{limit}. Wall-clock time since start: t. For Simulated Annealing, the temperature is T. For Tabu Search, the Tabu tenure is τ_{Tabu}. We use $\Delta = C_{\max}(\pi') - C_{\max}(\pi)$ for a candidate move.

Algorithm 1 GenerateAdjacent(π)

```
N ← ∅
for i = 1 to n − 1 do
    π′ ← π with positions i, i + 1 swapped
    N ← N ∪ {(π′, (i, i + 1))}
end for
return N
```

Algorithm 2 Composite Disjoint-Adjacent (*Fibonacci*) Neighborhood

```
base ← MAKESPAN(π); cand ← []
for i = 1 to n − 1 do
    π′ ← swap (i, i+1) in π
    Δ_i ← MAKESPAN(π′) − base
    append (i, Δ_i) to cand
end for
if cand empty then return (π, base, ∅)
end if
Sort cand by index
for k = |cand| down to 1 do
    Compute DP[k] = best tuple (value, chosenSet)
    Choose minimal value (ties: prefer more negative)
end for
(bestVal, chosen) ← DP[1]
if chosen = ∅ then
    Pick i* = arg min_i Δ_i; chosen ← {i*}
end if
π^new ← π
for each i ∈ chosen do apply swap (i, i+1)
end for
return (π^new, MAKESPAN(π^new), chosen)
```

Algorithm 3 DynasearchRecursive($\pi, depth$) (Sketch)

```
if depth = 0 then return (π, MAKESPAN(π))
end if
Decompose permutation into independent segments
best ← (π, MAKESPAN(π))
for each segment pattern and transformation θ do
    (π'', c'') ← DynasearchRecursive(θ(π), depth − 1)
    if c'' < best.c then best ← (π'', c'')
    end if
end forreturn best
```

Algorithm 4 TabuSearch($processing\ times, T_{limit}, \tau_{tabu}, neigMode$)

```
Initialize π, best, bestC, Tabu, iter, time
while t < T_limit do
    if neigMode = Adjacent then
        Candidates ← GenerateAdjacent(π)
    else if neigMode = Composite then
        (π', c', chosen) ← CompositeDisjointAdjacent(π)
    else if neigMode = Dynasearch then
        (π', c', _) ← DynasearchRecursive(π, depth)
    end if
    Update Tabu list, record best
end while
return (best, bestC)
```

Algorithm 5 SimulatedAnnealing($P, T_{limit}, T_{init}, T_{final}, \alpha, r, \tau_{stagn}, neigMode$)

```
Initialize π, c, best, bestC, T, timers
while t < T_limit do
    if neigMode = Adjacent then
        Pick random i; π' = swap (i, i+1)
    else if neigMode = Composite then
        (π', c', chosen) ← CompositeDisjointAdjacent(π)
    else if neigMode = Dynasearch then
        (π', c', _) ← DynasearchRecursive(π, depth)
    end if
    Apply acceptance rule, cooling, reheating
end while
return (best, bestC)
```

References

1. Ahuja, R.K., Orlin, J.B., Tiwari, A.: Very large-scale neighborhood search. Operations Research **50**(4), 686–702 (2002)
2. Blum, C., Roli, A.: Metaheuristics in combinatorial optimization: Overview and conceptual comparison. ACM Computing Surveys **35**(3), 268–308 (2003)
3. Bożejko, M., Bożejko, W.: Generalized gaussian processes and relations with random matrices and positive definite functions on permutation groups. Infinite Di-

mensional Analysis, Quantum Probability and Related Topics **18**(3), 1550020–1–1550020–19 (2015)
4. Bożejko, W., Wodecki, M.: On the theoretical properties of swap multimoves. Operations Research Letters **35**(2), 227–231 (2007)
5. Congram, P., Potts, C., Van de Velde, S.: An iterated dynasearch algorithm for the single-machine total weighted tardiness scheduling problem. In: Proceedings of the 3rd International Conference on Modeling and Applied Optimization. pp. 123–132. Springer (2002)
6. Deroussi, L., Gourgand, M., Norre, S.: New effective neighborhoods for the permutation flow shop problem. Tech. Rep. RR-06-09, LIMOS Research Report (2006)
7. Glover, F.: Future paths for integer programming and links to artificial intelligence. Computers & Operations Research **13**(5), 533–549 (1986)
8. Glover, F., Laguna, M.: Tabu Search. Springer (1997)
9. Johnson, S.M.: Optimal two- and three-stage production schedules with setup times included. Naval Research Logistics Quarterly **1**(1), 61–68 (1954)
10. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983). https://doi.org/10.1126/science.220.4598.671
11. Nowicki, E., Smutnicki, C.: A fast tabu search algorithm for the permutation flowshop problem. European Journal of Operational Research **91**(1), 160–175 (1996)
12. Serna, N.J.E., Gambardella, L.M.: A global-local neighborhood search algorithm and tabu search for the permutation flow shop problem. Computers & Operations Research **27**(11-12), 1167–1183 (2000). https://doi.org/10.1016/S0305-0548(99)00114-2
13. Smutnicki, C.: Job Scheduling Algorithms. Publishing House of Wrocław University of Science and Technology, Wrocław (2012), (In Polish)
14. Taillard, E.: Benchmarks for basic scheduling problems. European Journal of Operational Research **64**, 278–285 (1993)
15. Van Laarhoven, P.J.M., Aarts, E.H.L., Lenstra, J.K.: Job shop scheduling by simulated annealing. Operations Research **40**(1), 113–125 (1992)